

INTERNAL ENGINEERING DELIVERY PLAN

NEXTGENOS

A practical roadmap to one reliable business product and an evidence-based, governed intelligence platform.

REUSE THE CODE. UNIFY THE PRODUCT. PROVE THE OUTCOME.

PREPARED FOR ENGINEERING TEAM, QA, PRODUCT AND RELEASE OWNERS

8 October 2026 • NextGenOS strategy working draft

Based on the uploaded NextGenOS source archive, repository decision records and public competitor documentation. Future capabilities are explicitly labelled as planned.

1. Execution mandate

Deliver one dependable offline Business Hub that incorporates the strongest verified behavior of the legacy POS, AI companion and dashboard. Make retail/wholesale the first commercially validated vertical. Add a Palantir-inspired operational layer—real business entities, durable events, explainable decision logic and permissioned actions—without compromising financial correctness, local data ownership or cashier speed.

Design rule

Operational correctness beats AI novelty. Source-of-truth transactions stay in the Hub; intelligence reads authorized records and proposes typed, auditable actions. No model-generated direct database writes.

Binding product decisions (do not silently change)

- Reuse-first: inspect existing POS, companion, dashboard and website code before replacing or reimplementing any feature. Port behavior incrementally with tests; remove old code only after parity is proved.
- End-state: four customer/product groupings—Studio, unified offline software, website and Android app. The owner's Licence Studio is private within Studio and never shipped with signing keys.
- One main PC holds a store's authoritative data; counter devices pair over the shop's local network. Cloud transmission is off without explicit purpose-specific owner consent.
- Local scheduled backups to a second location; encrypted cloud backup only as opt-in. Signed updates proposed on main PC, owner approved, backed up first and reversible.
- Paid core licence: one-time/perpetual. Trials expire and stop selling while preserving access/export. Update/support inclusion, seats and add-on prices remain business decisions.
- External Studio design AI: public business branding details only. No customer records, transaction amounts, staff identity, secrets or payment data. Studio AI alters presentation, never backend logic.
- Payment provider capture is not in scope: record method/amount today; later certified provider integrations. Never store full card details.
- Do not delete history or source during parity work. Any later clean-repository migration needs explicit owner authorization and a verified archival/secret-rotation plan.

Source of record: docs/PLATFORM-DECISIONS.md (especially decisions 4, 7, 10–17 and 24–28), docs/MERGE-PLAN.md, docs/OPEN-WORK.md. This engineering plan is a proposal; repository decisions prevail where they differ.

2. Current-state map and deployment boundaries

Component / path	What exists	Boundary / gap
apps/business-hub/	Blazor web UI, .NET 10 domain services, SQLite, industry workflows, licence enforcement.	Single localhost store; not yet complete main-PC/counter-PC; financial/accounting gaps.

apps/pos-desktop/	Legacy WinForms retail/GST processes, reports and business rules.	Windows/SQL Server; merge selectively into Hub; do not expose as second shipped product.
apps/pos-ai-companion/	Existing AI assistance for legacy POS.	Legacy schema and different privacy/provider logic; refactor or adapt before integration.
apps/pos-dashboard-service/	Analytics and creative tools.	Separate program; unify useful workflows into Hub and approved services.
apps/storefront-web-mobile/	Product-discovery website and Android wrapper; independent Prisma/Postgres.	Not full checkout; no assumed real-time Hub stock; internet required for website app.
licensing/ + tools/setup-studio/	Signed entitlement system and customer-package assembly foundations.	End-to-end staff workflow, licence issuing, built-in kit and signing not complete.
country-packs/ + industry-packs/	34 country definitions, seven industry definitions and test vectors.	Not blanket legal compliance; packs are not self-contained runtime plugins.

Target high-level topology

Main-PC model

COUNTER UIs / approved devices → mutually authenticated LOCAL HTTPS → MAIN HUB APP → SQLite transactional store + outbox → background projections / rule engine → owner recommendations → approved action service. Optional cloud links live behind purpose-scoped opt-in adapters.

Separate the transactional system of record from derived analytics. At a single store, avoid introducing another mutable operational database. The ontology should map business records and relationships rather than duplicate financial values. Storefront/PostgreSQL has its own role and may consume opted-in, explicitly scoped publication snapshots. Multi-store head office receives optional aggregates only, not customer/staff/payment detail.

3. Non-negotiable technical invariants

1. Each issued sale has exactly one stable identity and bill number within its configured numbering scope. Retries never generate another sale for the same idempotency key.
2. Sale, payment, stock movement, loyalty, journal and outbox entry either commit together or roll back together.
3. Every posted journal entry balances in integer minor currency units; financial history is immutable or reversed through explicit entries.
4. A migration MUST abort if backup creation/validation fails; backups can be restored on a clean installation.
5. Counter PCs never keep independent writable copies of authoritative store state. A disconnected counter cannot silently allocate numbers or promise inventory it cannot verify.

6. No person or model can bypass server-side permission checks by invoking a service directly, skipping a page or replaying an old approval.
7. A paid licence's expiration warning behavior and a trial licence's stop-selling behavior are derived from signed entitlement policy; customers can still access/export their data where required.
8. No outside service receives shop data merely because a feature is installed. Consent, destination, data class and purposes are recorded and enforced.
9. Derived intelligence always records its input window, calculation version, timestamp, limitations and evidence links.

4. Phase 0 — baseline and defect containment (P0)

4.1 Create a reproducible branch and verification baseline

- Capture a source inventory, solution map, DB schema/migration version, release toolchain and all public API/entry points. Do not create a replacement application.
- Install supported .NET, Node and Windows build prerequisites on CI workers. Run node scripts/verify-all.mjs --full and record PASS/FAIL/NOT RUN per gate; do not infer Windows verification from Linux results.
- Create representative fixtures: retail cash sale, credit sale, returns, discounted tax, split payment, wholesale purchase, account credit and reopening a business database after crash.
- Open a tracked item for every discrepancy; tag it source-reproduced, observed-runtime, test-only or design-gap.
- Introduce release gate “no P0 data-integrity violations and no unresolved credential exposure” before shipping a pilot.

4.2 Fix migration pre-backup failure

Target: apps/business-hub/src/NextGenOS.Hub.Core/Data/HubDb.cs. Current Migrate() records BackupProblem on upgrade but can continue. Change migration preconditions so a noninitial schema upgrade fails closed if consistent snapshot creation or verification fails. Add a restore drill and disk-full, inaccessible-backup-folder and interrupted-migration tests. Preserve original DB until upgrade success; do not mutate in failure paths.

Acceptance: DB-001

With the backup destination unwritable, a noninitial Migrate() throws an actionable error, writes no migration rows and leaves schema/data checksums unchanged. A valid backup restores and passes PRAGMA integrity_check + domain invariant checks.

4.3 Secure secrets and access boundaries

Audit secrets in current files, released artifacts and actual Git history when available; rotate any previously exposed credential. Review every privileged Hub.Core service entry path. Enforce authorization at the command boundary, with explicit actor, store and permission context; UI role checks remain supplementary. Review personally identifiable information masking in the old AI companion and remove inappropriate remote data flow.

5. Phase 1 — reliable finance, stock and retail parity (P0/P1)

5.1 Financial correctness

Target: Documents/DocumentService.cs, Books/BooksService.cs, purchasing, returns, loyalty and journal migrations. Keep integer minor units and share tax calculation definitions. Implement explicit inventory valuation policy (e.g., weighted average or FIFO as a product/accounting decision), immutable receipt cost basis, stock movement costing and COGS. Define posting entries for cash/credit sale, return, purchase/receipt, supplier payment, discount, shrinkage and period close. Backfill old data through opening balances rather than recalculating historic bills.

Test scenario	Required observable result
Discounted taxable sale → partial return	Tax allocations and refund match the configured rule; no rounding drift beyond documented minor-unit allocation.
Credit sale → on-account payment → return	Customer ledger, invoice view and journal agree, including stored-credit treatment.
Purchase receipt → sale → stock correction	Stock quantity, valuation, COGS and journal balance reconcile.
Repeated request / process crash	Exactly one finalized financial transaction; no duplicate payment, loyalty or stock change.
Reopen after migration and restore	Trial balance, outstanding balances and inventory value match approved reconciliation totals.

5.2 Legacy POS feature mapping

Use docs/MERGE-PLAN.md and docs/old-programs/ as the behavioral specification, then update them against current code. Create a parity register for each old workflow: old screen, business behavior, hidden tax/credit side effects, destination service/UI, fixtures, customer priority, owner choice on intentional changes, passing tests and retirement decision. Reuse source logic where safe; do not mirror known broken practices.

- First wave: checkout/discount/tax modes, customer and supplier ledgers, coupons and vouchers, stock adjustments, price/variant semantics and essential GST reports for India only after local review.
- Second wave: purchase returns, barcode labels, stock transfers, day/cash/bank books and loyalty financial liability.
- Later waves: other niche reports, complex offers, advanced staff workflow, integrations and industry-specific additions based on verified demand.

5.3 Migration path

Build read-only connectors for accessible SQL Server exports first, then file/CSV inputs and additional supported sources. Preserve source identifiers, map ambiguous legacy columns explicitly, and generate a dry-run report comparing counts, daily sales, stock, ageing and party balances. Require authorized sign-off before cutover. Support retry/resume via imported-source keys; never alter the original system. Records that cannot be imported must be explained, not silently skipped.

Acceptance: MIG-001

A representative legacy SQL Server fixture imports in read-only mode, produces a signed/dated reconciliation report, and either matches all required money totals or blocks go-live with an itemized difference report.

6. Phase 2 — one secure main PC, multiple counter PCs (P0)

Current Hub binds to 127.0.0.1:5280 and serves a single local PC. Deliver a documented local-network topology with the store main PC hosting the authoritative application and database, and counters using authenticated clients over local TLS. Avoid peer-to-peer multi-master sync in this phase: the owner has chosen a single main PC.

Security and connectivity contract

- Generate per-shop certificates/keys on the main PC; pair counters through a one-time code or QR flow with explicit owner approval. Never share private keys with the Studio or cloud.
- Enforce device identity, user authentication, role permissions, expiry/revocation and replay protection; local TLS must validate peer identity.
- Expose an authenticated LAN endpoint, not an unauthenticated SQLite file share. Create a single-writer policy and test SQLite busy/retry behavior under simultaneous sales.
- Disconnected counters enter explicit read-only/retry states for financial actions; display queue status and do not create shadow bills.
- Record device identity and user in audit events; provide a revoke-device workflow and help for changing the main PC.

Concurrency/chaos test matrix

Stress case	Required outcome
Two counters sell the last unit together	Only valid authorized transaction(s) succeed; no negative stock unless policy explicitly permits it.
25 concurrent issue requests	Unique document numbers; no SQLite busy failure escapes as a corrupted/partial transaction.
Network drops after server commits	Client retries safely with same idempotency key and receives original result.
Main PC reboots during busy day	Recovery works; counters see accurate reconnect state, no duplicate invoices.
Pairing key revoked	Revoked counter can no longer call protected operations.

7. Phase 3 — real business events and evidence (P1)

7.1 Implement a transactional outbox

Keep HubDb as the authoritative transaction boundary. Add a durable outbox table written within the same database transaction as a sale or other business command. Do not publish directly to in-memory subscribers from a service before commit. A hosted dispatcher reads committed rows and retries safely; derived consumers keep unique processed-event IDs.

Proposed event envelope (illustrative, not an existing schema)

```
event_id UUID | store_id | aggregate_type/id | event_type/version | occurred_at UTC | actor_id |
correlation_id | causation_id | transaction_id | data_class | payload_minimal | evidence_refs |
schema_version
```

Initial events: sale.issued.v1, sale.voided.v1, sale.returned.v1, stock.adjusted.v1, purchase.received.v1 and payment.recorded.v1. Events must contain necessary identifiers and limited facts; no raw card detail, secrets or arbitrary customer notes. Store current business state in normal domain tables and the event history as evidence. Correction must be explicit.

7.2 Outbox algorithm

1. Write business changes + outbox message inside the same SqliteTransaction.
2. After commit, dispatcher acquires a lease on pending rows; bounded batch + backoff + terminal failure state.
3. Dispatch to event store or projection handler with idempotent consumption and recorded schema version.
4. Only mark delivered after the handler confirms persistence; tolerate at-least-once delivery.
5. Expose owner diagnostics for backlog length, oldest unprocessed age, failures and replay status.

Acceptance: EVT-001

Kill the process after committing a sale but before dispatch: restart delivers exactly one effective projection update. Redelivering the same event cannot duplicate sales, alerts or action proposals.

8. Phase 4 — ontology as a trusted operational view (P1)

Targets: Hub.Core/Ontology/OntologyService.cs, Ontology/Catalogue.cs and business entity adapters. Preserve stable typed IDs (e.g., product:<id>, supplier:<id>, invoice:<id>, zone:<id>), foreign-key context, deletion/retirement semantics and evidence. The ontology is an authorized read model; it must not become a new writable source for money/stock.

Minimum graph and time contract

Object types	Relationships / reason to connect
Product; StockLocation; Supplier	Stock held at location; supplied by vendor; lead time and

	cost references.
Invoice; SaleLine; Payment; Customer	Which product sold, to whom, with what payment or balance.
PurchaseOrder; GoodsReceipt; Supplier	Order proposal to actual delivery, timing and exception outcomes.
Employee; Store; Device; Zone	User/device/physical context with granular access checks.
Recommendation; Evidence; ApprovedAction	What was suggested, why, who approved and outcome measured.

Include as_of timestamps, source reference and derived-vs-authoritative labels. Resolve live sensitive fields through the Hub's permissioned query layer. Filter nodes, edges, properties and evidence using caller role and data classification; permission checks must occur before returning data to AI or web UI.

Intelligence-ready query examples

- Products with less on-hand cover than supplier lead time, limited to authorized locations.
- Customers with overdue balances and valid communication preferences.
- Orders that were suggested, approved, delivered, and whether they prevented a stockout.
- Stock movements by reason and approved actor, without leaking staff or customer data to unauthorized roles.

9. Phase 5 — deterministic intelligence + governed actions (P1)

9.1 Launch rules before general-purpose agents

Recommended first rules: low stock/reorder, slow-moving inventory, overdue receivables, margin anomalies and unusual returns for manager review. Calculations must run over reliable source-of-truth numbers with explicit currency, stock units, time windows, missing-data handling and threshold configuration. Backtest rules before making claims about savings.

Low-stock rule (illustrative)

$\text{stock_cover_days} = \text{on_hand_units} / \text{avg_daily_units_sold}$ (when rate > 0). Flag when cover is below verified supplier lead time + configured safety days. A proposed reorder quantity may account for open purchase orders, minimum quantities, case packs and recent demand; present assumptions for review.

9.2 Typed action registry

Create a service inside existing Hub.Core (suggested namespace: Actions) rather than a new standalone program. A registered action binds a versioned input schema, required entitlements, user permissions, validations, dry-run result, executable domain command, audit record and idempotency logic.

Action ID

Approvals / core guardrails

CreatePurchaseOrder.v1	Purchasing permission, current supplier/stock check, owner or manager approval, duplicate draft detection.
AdjustStock.v1	Stock permission, reason code, second approval above configured thresholds.
RecordCustomerPayment.v1	Financial permission, no phantom payment confirmation or automatic external collection.
PreparePaymentReminder.v1	Consent and communication preferences; default draft only; no remote sending without owner approval.
ApproveDiscount.v1	Discount cap, cashier/manager policy, explicit audit trail.

9.3 Action state machine

Proposed status progression

PROPOSED → VALIDATED → AWAITING_APPROVAL → APPROVED → EXECUTING → SUCCEEDED / FAILED / CANCELLED / EXPIRED. Every transition records actor, permission, reason, inputs and source evidence; the execution rechecks invariants at commit.

A user's approval cannot be reused after significant changes to price, amount, quantity, destination, supplier, account balance or expiry. External side effects require compensation policy, retries and a durable dispatch queue. Never provide the LLM a generic "execute SQL" or "call arbitrary URL" action.

9.4 Explainability and outcome loop

Each recommendation carries rule_id/version, evidence, as_of, confidence if statistically meaningful, assumptions, suggested action and "why might this be wrong?". Track accepted/rejected recommendations and actual outcomes. Monitor nuisance alerts, false positives and action cancellations, not just click-through rates.

10. AI gateways, consent and privacy controls (P0/P1)

There are two distinct consent regimes. Studio design assistance may send only public branding material with contractual agreement; it must never send shop transactions or change backend code. Hub AI starts off and is optional; any remote processing must follow feature-specific owner permissions and the strictest applicable classification rules. Never send payment card details or biometric data to hosted providers.

Implement enforceable data egress

- Create a single classification vocabulary: PUBLIC_BRAND, SHOP_OPERATIONS, CUSTOMER_PERSONAL, EMPLOYEE_PERSONAL, PAYMENT_SENSITIVE, SECRETS, BIOMETRIC. Map to existing repository policy identifiers as appropriate.
- For every remote AI call, log model/provider, consent version, purpose, allowed class, field-level payload summary, cost and outcome; do not log the raw sensitive payload.
- Use typed tool outputs with a fixed allowlist of fields. Block arbitrary table dumps, customer notes and model-supplied SQL.

- Preserve local-only mode; use remote AI only when explicitly enabled and supported by privacy review.
- Move critical queued AI jobs from volatile memory to a durable store when jobs must survive reboot; hard-limit cost and concurrency.
- Add tests injecting names, addresses, free-text remarks, payment identifiers and adversarial prompt content to ensure prohibited data never reaches remote adapters.

Security tests

Add authorization unit tests on domain commands; integration tests for revoked devices; API-level negative tests for cross-store access; key-storage tests on real target OS; dependency and secrets scans in release CI; package integrity/signature verification; explicit consent-revocation tests. Existing secret findings in docs/OPEN-WORK.md require actual credential rotation and history review in the live repository.

11. Industry packs: expand coverage without multiplying products

Keep industry-packs/ as versioned manifest inputs and Hub.Core as shared infrastructure. An industry pack is not a standalone full application: workflows require compiled, tested services. Start with retail and wholesale readiness; other industries remain validated baselines rather than a blanket go-live guarantee.

Recommended pack contract, version 1

Manifest section	Purpose / validation rule
pack_id; pack_version; min_core_version	Identity, compatibility and upgrade policy.
required_features; optional_features	Entitlement checks, dependency resolution and downgrade behavior.
navigation; screen_presets; terminology	Role-aware UI composition; translations cannot alter domain logic.
workflow_ids; action_ids	Explicitly registered implementations; fail if a declared handler is absent.
ontology_types; rule_ids; report_ids	Typed extension schemas, permission mapping and provenance.
country_constraints; regulatory_claims	Jurisdiction-specific review labels; no automatic “tax compliant” marketing.
migrations; fixtures; acceptance_tests	Upgrade safety, industry sample data and pass/fail certification.

Avoid building an unrestricted runtime code marketplace in phase one. Prefer signed, versioned compiled modules or a restricted declarative extension format. Industry-specific business rules must never be editable by Studio AI styling tools.

12. Storefront, Android, head office and Studio integration

12.1 Website and Android

Current storefront is product discovery/enquiry and explicitly does not offer online checkout. Keep that boundary in marketing and integration scope. Define an owner-approved publication adapter that exports only chosen catalogue fields and coarse availability if appropriate, with as_of timestamps. PostgreSQL remains web-content storage, not authoritative stock or finance. The Android wrapper currently depends on an online website.

12.2 Optional multi-store reporting

Each store remains independent and local-first. A future owner-enabled relay may publish aggregate sales, stock levels and takings to head office. Do not publish identities, raw payments, personal information or unapproved operational records. Include visibility, revocation and deletion in the consent UX.

12.3 Studio, licensing and delivery

- Staff selects an industry, country, store layout and purchased entitlements; preview exactly what the customer will receive.
- Licence issuance happens through the owner-controlled Licence Studio; private keys never reach customer/staff bundles.
- Assemble one tested base kit per release; verify every artifact fingerprint and sign deliverables; trial vs perpetual license policy is encoded in signed claims.
- Studio AI can modify approved appearance values/presentation-only styling and supports review, undo and validation; it cannot modify backend behavior.
- Customer sees update offer on main PC and approves; pre-update backup and rollback are mandatory. Apply signed updates to locally paired counters.
- Data import/export and signed support diagnostics are available without silently uploading the store database.

13. Backlog: build-ready tickets and definition of done

Use tickets below as the first executable backlog. Estimates and named owners should be added by the engineering lead after repository-level code review; none of these should be described as already delivered.

ID / priority	Task and target	Acceptance proof
DB-001 / P0	Fix HubDb.Migrate pre-backup behavior.	Fail-closed migration test + restore drill.
OPS-002 / P0	Automated nightly local encrypted-or-permitted backup to second location; restore UI.	Scheduled run, last-good status and unplugged-destination failure tests.
FIN-003 / P0	Journal/customer invoice consistency and valuation/COGS plan.	Accounting vector suite and trial-balance reconciliation.
SEC-004 / P0	Command-boundary RBAC and cross-store isolation.	Negative authorization tests from non-UI callers.

SEC-005 / P0	Rotate leaked credentials; audit actual Git/artifact secrets.	Revocation evidence and clean scanner output.
NET-006 / P0	Authenticated main-PC/counter pairing.	LAN/TLS tests on two physical PCs.
NET-007 / P0	Concurrent billing + retry/idempotency.	25 concurrent issues + disconnect/retry suite.
MIG-008 / P0	Read-only legacy SQL Server pilot and reconciliation.	Realistic dry-run, matching totals or blocked cutover.
EVT-009 / P1	Transactional outbox + idempotent dispatcher.	Crash recovery and replay tests.
ONT-010 / P1	Stable typed ontology identities + ACL.	Graph correctness and least-privilege query tests.
INS-011 / P1	Low-stock / supplier-lead-time rule and evidence.	Reproducible deterministic snapshots and threshold tests.
ACT-012 / P1	Typed action registry + approval/audit.	Role/expiry/replay/no-approval negative tests.
ACT-013 / P1	Draft purchase-order approved action.	Full cashier-to-order acceptance test.
AI-014 / P1	Field-level egress control and consent audits.	PII injection/remote payload capture tests.
LIC-015 / P1	Paid perpetual/trial expiry behavior across clients.	Signed spec + cross-language vectors + end-to-end tests.
REL-016 / P1	Signed owner-approved updates with rollback.	Pilot Windows install/update/restore/revert.
KIT-017 / P1	Base kit + Studio/licence customer assembly.	Two distinct customers from one release with isolated brands/keys.
WEB-018 / P2	Opt-in published product snapshots with freshness.	Visibility/consent tests; no checkout or stock ownership confusion.
QA-019 / P0	Real hardware and platform certification matrix.	Printer/scanner, Windows service, offline and OS restart evidence.
GT-020 / P1	Pilot telemetry and customer-success dashboard (privacy safe).	Baseline and post-pilot measures without covert data upload.

Global definition of done

- Code lives in the appropriate existing project and has ownership, design notes and tests; no duplicate new application without explicit approval.
- All affected money, stock, authorization and licence tests pass, including negative and recovery cases.

- User-facing changes work offline if they are core shop operations, and behave predictably under main-PC failure.
- Feature flags, privacy, consent, permissions and audit events are tested at the service boundary.
- Customer documentation, migration notes, backup/restore instructions and QA evidence are updated.
- Artifact passes full release/protection/licence/install gates on supported target operating systems.

14. Proposed delivery waves and measurable gates

Wave	Work focus	Do not proceed until...
0 / baseline	CI parity, urgent DB defect, credential rotation, core access audit.	Required check suite has results and no unresolved data-integrity PO.
1 / reliable shop	Retail finance, backup/restore, SQL import reconciliation and hardware.	Single shop completes sample day and restore without loss/corruption.
2 / local multi-counter	Pairing, TLS, service authorization, concurrency and recovery.	Multiple counters pass load and interruption tests.
3 / operational intelligence	Events, ontology, low-stock rules, evidence and owner UX.	Alerts replay deterministically and contain no unauthorized data.
4 / governed action	Action registry and manager-approved purchase-order drafts.	Approval, audit, stale proposal and duplicate-action tests pass.
5 / distribution	Licence policy, Studio kit, signed update, pilots and partner readiness.	New customer onboarding is repeatable; pilots show measurable value.

Pilot scorecard

Metric	Baseline method	Passing signal
Billing reliability	Count failed/duplicated transactions during controlled run.	Zero unresolved duplicate or partial financial transactions.
End-of-day reconciliation	Compare counter totals, journal, stock and receivables.	All material totals explainably reconcile.
Backup/recovery	Restore on clean PC from last scheduled backup.	RTO/RPO targets chosen by pilot owner and actually met.
Counter speed	Time realistic basket checkout on target hardware.	At least matches agreed existing-system baseline.
Insight trust	Manually recompute sample low-stock alerts.	All audited alert arithmetic is reproducible.
Onboarding effort	Log time from blank device to trained shop use.	Repeatable effort declining across similar pilots.

Customer value	Capture measured saved time, fewer stockouts or collection improvement.	At least one outcome that pilot owner agrees is worth paying for.
----------------	---	---

15. Architecture decisions required before deeper coding

These are implementation choices the technical lead must propose with a brief ADR and obtain the owner's decision where product behavior is affected:

1. Inventory valuation method, opening-stock handling and period closing rules.
2. Whether the first supported counter is only a browser client or also a packaged desktop counter shell.
3. The network pairing/certificate issuance and device revocation UX.
4. Official support/update scope bundled into the initial perpetual purchase; extra device/module pricing.
5. Consent UX and precise data classes for optional Hub remote AI and product publication.
6. Which legacy workflows form the minimum retail/wholesale launch definition.
7. Country-by-country tax/compliance marketing and which regulators/local experts review claims.

16. First 10 engineering tasks (in practical order)

1. Run and record the full available test matrix on correctly provisioned Windows and Linux environments. Publish honest not-run items.
2. Fix pre-migration backup abort behavior and add restore validation.
3. Capture source-of-truth finance, stock and licence invariants as executable tests.
4. Rotate credentials and remove privileged service operations from UI-only trust.
5. Design nightly local backups and a one-click restore UX.
6. Prepare a read-only SQL Server fixture and reconcile one realistic retail migration.
7. Load-test concurrent checkout on Hub SQLite before exposing it on LAN.
8. Add secure LAN pairing and local TLS, then two-counter acceptance tests.
9. Build a durable outbox for sale/return/stock/payment events with replay tests.
10. Use those events to create one evidence-backed low-stock recommendation and an approved purchase-order draft.

First commercially convincing milestone

A real pilot shop can sell safely on the main PC and counter PC, recover its records, reconcile money and stock, and approve a source-backed purchase-order draft—with no hidden cloud data transfer.

17. Source map and research reference

Repository files to review before implementation

- docs/PLATFORM-DECISIONS.md — owner's binding product and pricing decisions.
- docs/MERGE-PLAN.md; docs/old-programs/ — older feature inventory and business-rule traps.
- docs/OPEN-WORK.md; docs/V2-ARCHITECTURE-ASSESSMENT.md — known limits and AI roadmap.
- apps/business-hub/src/NextGenOS.Hub.Core/Data/HubDb.cs — migrations, backup and SQLite transactions.
- apps/business-hub/src/NextGenOS.Hub.Core/Documents/DocumentService.cs — sale issuance and shared money paths.
- apps/business-hub/src/NextGenOS.Hub.Core/Books/BooksService.cs — journal and accounts.
- apps/business-hub/src/NextGenOS.Hub.Core/Events/EventStore.cs — evidence/event foundation.
- apps/business-hub/src/NextGenOS.Hub.Core/Ontology/{Catalogue.cs,OntologyService.cs} — object/link model.
- apps/business-hub/src/NextGenOS.Hub.Core/Ai/ — provider gateway, privacy and local-job infrastructure.
- apps/business-hub/e2e/ — cross-industry browser tests; apps/business-hub/tests/ — domain tests.
- licensing/spec/LICENCE-FORMAT.md — signed licence schema and test vectors.
- apps/storefront-web-mobile/prisma/schema.prisma — storefront independent data model.
- country-packs/; industry-packs/; libs/dotnet/NextGenOS.Devices/ — configuration and peripheral contracts.

External architecture and competition references

- Palantir, platform overview (conceptual inspiration only): <https://www.palantir.com/docs/foundry/platform-overview/overview/index.html>
- Palantir, action types (conceptual inspiration only): <https://www.palantir.com/docs/foundry/action-types/overview/index.html>
- Odoo 19 POS offline operation and limitations: https://www.odoo.com/documentation/19.0/applications/sales/point_of_sale/use.html
- Zoho POS India features/pricing comparator: <https://www.zoho.com/en-in/pos/pricing.html>

The above is a proposed execution blueprint based on source inspection and owner decisions, not a statement that every described system, acceptance test, screen, integration or deployment is already present. Any code changes require a fresh branch-level read, compilation and executable validation.